

Data Handling and Infrastructure for AI

Data Storage

Tamás Grósz

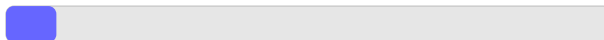
SETU

2026



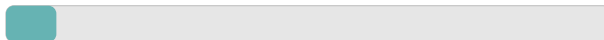
Workload

Lectures



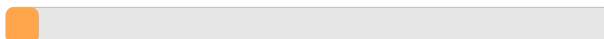
2 / 24 hrs

Labs



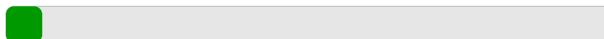
2 / 24 hrs

Self-Study



11 / 202 hrs

Overall Progress



15 / 250 hrs

Intended Learning Outcomes

By the end of this lecture, you should be able to:

- **Compare Processing Paradigms:** batch vs. streaming data in terms of latency, throughput, and suitability for AI training versus real-time inference.
- **Design Robust Data Splits:** implement Train/Dev/Test or Cross-Validation (CV) partitions and prevent data leakage.
- **Optimize Storage & I/O:** identify key I/O bottlenecks and choose an appropriate storage solution.

Why Does Storage Matter for AI?

A modern AI system is often limited by **data movement**, not computation.



- GPUs can process enormous amounts of data per second.
- If data cannot be delivered fast enough, the accelerator sits idle.
- Poor storage choices can therefore make an expensive AI system slow.

Key question

How do we move the right data to the right computation at the right speed?

Storage Hierarchy

Data can live at very different levels of the storage hierarchy.

Storage	Typical property	AI example
CPU cache	Extremely fast, tiny	Repeated computation
RAM	Fast, limited	Data preprocessing
Local SSD	Fast, persistent	Training dataset cache
Network storage	Shared	Team / cluster datasets
Object storage	Large, scalable	Data lake / raw data

Trade-off:

speed ↔ capacity ↔ cost

The I/O Bottleneck

Suppose a GPU can perform training much faster than data can be loaded.

Storage $\xrightarrow[\text{too slowly}]{\text{I/O}}$ Data Loader $\rightarrow$ GPU

GPU utilization ↓

Common bottlenecks include:

- Slow storage device
- Network bandwidth
- High I/O latency
- Too many files
- Expensive serialization / deserialization
- CPU preprocessing becoming the bottleneck
- Poor data loading parallelism

Why Batch Processing?

A major motivation for batch processing comes directly from how many ML algorithms train. For example, in **Stochastic Gradient Descent (SGD)**:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(f_{\theta}(x_i), y_i)$$

Rather than processing the entire dataset at once, we typically process a **mini-batch**:

$$B = \{(x_1, y_1), \dots, (x_m, y_m)\}$$

and estimate:

$$\nabla_{\theta} L \approx \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L_i$$

Infrastructure implication

Training requires repeatedly reading large amounts of data in manageable batches, making efficient sequential and parallel I/O critical.

Batch Processing

Batch processing collects data and processes it as a group.



	Batch	Typical example
Latency	Higher	Minutes / hours
Throughput	High	Large datasets
Typical use	Training	Model training
Advantage	Efficient bulk processing	

Examples:

- Nightly training jobs
- Preparing training datasets
- Large-scale feature generation
- Periodic model retraining

Streaming Processing

In streaming systems, data is processed as it arrives.

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4 \rightarrow \dots$$


Processing continuously

- **Low latency**
- Continuous data ingestion
- Suitable for real-time decisions
- Often requires state management
- Individual events may be small

Batch vs. Streaming

	Batch	Streaming
Latency	High	Low
Throughput	Very high	High
Data arrival	Periodic / accumulated	Continuous
Typical AI use	Training	Inference / monitoring
Storage pattern	Large files / datasets	Events / messages
Complexity	Relatively lower	Higher

Important

These are not mutually exclusive.

A real AI system may use streaming ingestion, batch processing for training, and streaming inference.

Streaming → Online and Continual Learning

Streaming data can also change how models are trained.

Online learning:

$$(x_t, y_t) \rightarrow \text{update model} \rightarrow (x_{t+1}, y_{t+1}) \rightarrow \dots$$

The model is updated incrementally as new observations arrive.

Continual learning considers the longer-term problem of learning from a changing stream of data.

- Data distributions may change.
- New concepts may appear.
- Old knowledge may become less relevant.
- The model may need periodic or continuous updates.

Infrastructure challenge

How do we safely update a production model while new data keeps arriving?

The Many Small Files Problem

Imagine a dataset consisting of:

10 000 000 files

where each file contains only a few KB.

The problem is not necessarily the total amount of data.

- Every file has metadata.
- Opening a file has non-zero latency.
- Network filesystems incur many requests.
- Parallel workers can overwhelm the storage system.

Many small files → many I/O operations → poor throughput

Solving the Small Files Problem

Instead of storing:

$\text{image}_1, \text{image}_2, \dots, \text{image}_{1M}$

we can package data into larger **shards**, containing many examples.

Shard ₁	Shard ₂	Shard ₃	...
--------------------	--------------------	--------------------	-----

Benefits:

- Fewer filesystem operations
- More sequential I/O
- Easier parallel data loading
- Better utilization of distributed storage

Common approaches include:

- Sharded archives
- TFRecord
- WebDataset / TAR shards
- Parquet
- Other large, chunked binary formats

Data Formats Matter

The same information can have very different I/O characteristics.

Format	Strength	Typical use
CSV	Simple, portable	Small tabular data
JSON	Flexible / nested	APIs, metadata
Parquet	Columnar, compressed	Analytics / data lakes
TFRecord	ML-oriented records	TensorFlow pipelines
TAR shards	Simple sequential access	Large image datasets

Rule of thumb

Choose a format based on the access pattern — not simply because it is familiar.

Databases: SQL vs. NoSQL

Databases are another important part of AI infrastructure.

SQL

- Relational tables
- Schema
- SQL queries
- Strong consistency / transactions
- Joins and analytical queries

Examples:

- PostgreSQL
- MySQL

NoSQL

- Flexible data models
- Key-value / document / wide-column
- Horizontal scalability
- Often designed for high-volume workloads

Examples:

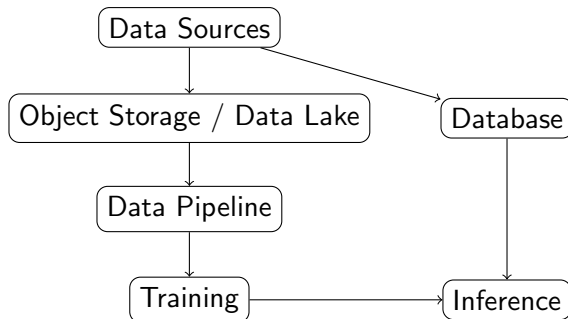
- MongoDB
- Cassandra

Question

Do we need transactions and relationships, or enormous scale and flexible schemas?

Where Should Data Live?

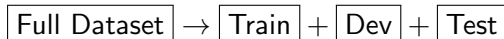
A simplified AI data architecture:



Different components have different storage requirements.

Training Data: Train / Dev / Test

A dataset should generally be partitioned before model development.



- **Train:** fit model parameters.
- **Dev / Validation:** choose models and hyperparameters.
- **Test:** final, unbiased performance estimate.

Example:

70% / 15% / 15%

Critical principle

The test set should not influence model development decisions.

Cross-Validation

When data is limited, we can use **K-fold cross-validation**.

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Test	Train	Train	Train	Train
Train	Test	Train	Train	Train
Train	Train	Test	Train	Train
Train	Train	Train	Test	Train
Train	Train	Train	Train	Test

Each fold is used as validation once.

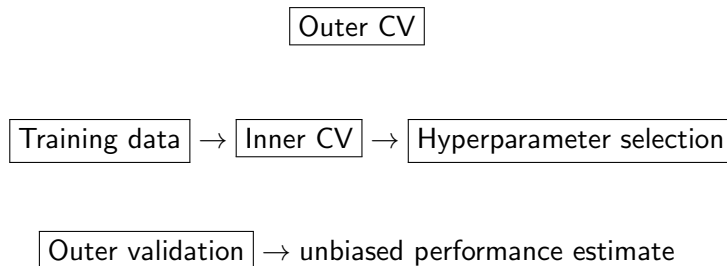
$$\text{CV score} = \frac{1}{K} \sum_{k=1}^K \text{score}_k$$

This provides a more robust estimate when the dataset is relatively small.

Nested Cross-Validation

What happens when we use CV to **select hyperparameters**?

We need to avoid using the same data to both select and evaluate the model.



Nested CV therefore has:

- Inner loop: model / hyperparameter selection
- Outer loop: performance estimation

Data Leakage

Data leakage occurs when information that should be unavailable during training influences the model.

Example:

Full dataset → Normalize → Split

This can leak information from the validation/test sets into training.
Instead:

Split → Fit preprocessing on Train → Transform Train / Dev / Test

Other examples:

- Duplicated samples across train and test
- Future information in time-series features

• Target-derived features

Distributed Training and Data Loading

Large models and datasets often require multiple machines or GPUs.
A common approach is **Distributed Data Parallel (DDP)**.

$$\text{Dataset} \rightarrow \begin{cases} \text{GPU 1} \rightarrow \text{Batch}_1 \\ \text{GPU 2} \rightarrow \text{Batch}_2 \\ \text{GPU 3} \rightarrow \text{Batch}_3 \\ \text{GPU 4} \rightarrow \text{Batch}_4 \end{cases}$$

Gradients $\rightarrow$ Synchronize $\rightarrow$ Model update

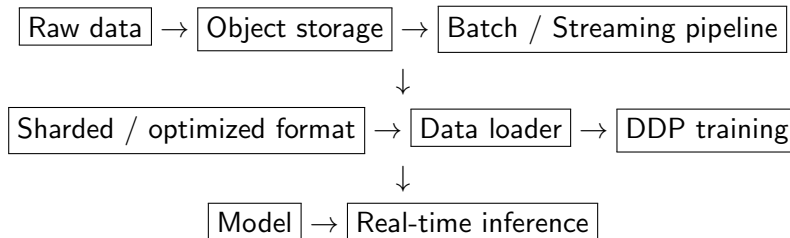
DDP can increase training throughput, but it also increases the importance of efficient data loading and storage. (Sharded data works quite well with DDP)

Infrastructure connection

More GPUs $\neq$ faster training if the storage system cannot feed them.

Putting It Together

A scalable AI data pipeline might look like:



At every stage we need to ask:

- How much data?
- How quickly does it need to move?
- Who needs access?
- What consistency guarantees are required?
- How do we prevent leakage?

Version Control for Data

Git works extremely well for source code, but datasets can quickly become too large for a normal Git repository.

Code → Git

Large datasets → Git LFS / Data Versioning

Git Large File Storage (Git LFS) keeps large files outside the normal Git object database while Git tracks a lightweight pointer.

- Git repository contains metadata / pointers.
- Large files are stored separately.
- Dataset versions can be associated with code versions.
- Useful for smaller-to-medium datasets and ML artifacts.

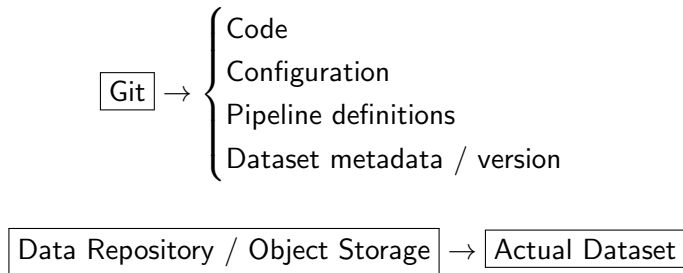
Instead of storing datasets directly in Git, we can use specialized dataset repositories.

Popular options include:

- **Hugging Face Datasets** — datasets for NLP, vision, audio and multimodal AI.
- **Kaggle Datasets** — datasets and competitions across many application areas.
- **OpenML** — datasets, experiments and ML benchmarks.
- **Zenodo** — research datasets and other research artifacts, with persistent identifiers.

Dataset Versioning

A useful project structure is:



Best practice

Don't commit a multi-GB dataset into ordinary Git just because “it works” — separate **code**, **data**, and **data metadata**, while keeping their versions linked.

- **Storage is part of the AI system**, not merely a place to keep files.
- **Batch processing** provides high throughput and is particularly important for large-scale training and ETL.
- **Streaming** provides low latency and enables real-time inference, online learning, and continual learning.
- **Sharding and efficient formats** help solve the many-small-files problem and improve I/O throughput.
- **SQL and NoSQL** databases address different access and scalability requirements.
- **Train/Dev/Test and CV** must be designed carefully to prevent data leakage.
- **DDP and distributed training** make efficient data loading increasingly important.

What storage architecture would you choose?

Consider an AI system with:

10 TB of training data

+ millions of images + real-time user events

Where would you store it?

How would you process it?

How would you feed it to the GPUs?

Please use Slack for the discussion